



WORKSHOP

Getting Started with ML in Production

From notebook to a live, monitored API — a hands-on, 2-day path to shipping your first ML model.



FACILITATOR

Jimmi Kr. Bharti

[in/jimmi-kr](https://www.linkedin.com/in/jimmi-kr) ↗

2 DAYS · 3 HOURS / DAY · BEGINNER FRIENDLY

What This Deck Covers

Switch back to this deck anytime between hands-on exercises.



The Big Picture

What actually happens end-to-end when an ML model goes to production



Day 1: Intro & API

Packaging a model and building your first working FastAPI, all running locally



Day 2: Ship It

Deploying your API live on Render, and watching real requests come in



The Bigger Picture

Everything else the industry does — named and explained, nothing to build



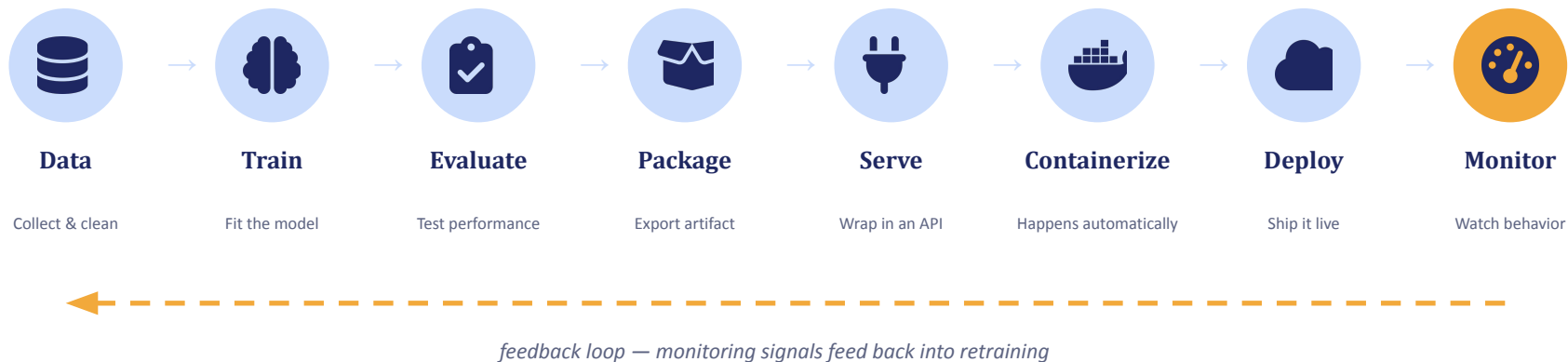
01

The Big Picture

What actually happens, end-to-end, when an ML model goes into production

The End-to-End ML Production Pipeline

Every production ML system — simple or complex — moves through these same stages.



You will build **Package** → **Serve** → **Deploy** → **Monitor** by hand across these two days. Containerize happens automatically when you deploy — more on that on Day 2.

What You'll Walk Away With

By the end of these 6 hours, you will have personally done every one of these — not just watched a slide about it.



Exported a trained model as a reusable artifact



Built a REST API that serves live predictions



Deployed it to a public URL anyone can call



Watched real requests in a live log, including a failed one



Pushed a change and confirmed it went live



Explained the full pipeline — including the parts you did not build by hand



What this workshop is NOT

It won't make you an MLOps expert, cover AWS/Azure/GCP-specific services, or have you build Docker containers, CI/CD pipelines, or versioning systems by hand. Those are named and explained on Day 2, but not built. That is a deliberate scope choice, not a shortcut.

03

Day 1 — Intro & Your First API

Packaging a model and building a working FastAPI, entirely on your own machine

Day 1 Roadmap

3 hours · 5 blocks · 10-min break after Packaging · ~55 min built-in buffer

10 min



Setup Check

Confirm Python and the repo work before we build anything. No Docker needed today.

15 min



Why Production Is Different

Notebook vs. live system, plus a quick demo of a deployed model.

35 min



Packaging Your Model

A trained model is just data. Save it to a file, load it back. Hands-on: export model.pkl.

50 min



Building Your First API

APIs explained with a restaurant. Two small wins in FastAPI: health check and prediction.

5 min



Checkpoint

Confirm everything works before Day 2 builds on it.

Setup Check

Three quick checks. If all three work, you are ready for everything else today.

1

Check Python

You need version 3.10 or higher. (Mac/Linux: use python3.)

```
python --version
```

2

Install the libraries

Run this from the repo's main folder. Takes a minute or two.

```
pip install -r requirements.txt
```

3

Open Jupyter

A browser tab should open. Leave it open for the next session.

```
jupyter notebook
```

Tip: Stuck on any step? Raise your hand now. Fixing setup early is far cheaper than fixing it mid-session. Nobody needs Docker installed today.

Demo: Same Model, Two Worlds

Live: I will call a deployed model from my phone. By tomorrow, you will have your own.



In the notebook

- Runs only on my laptop
- Only I can use it
- Stops the moment I close the notebook
- Input is typed into a cell by hand



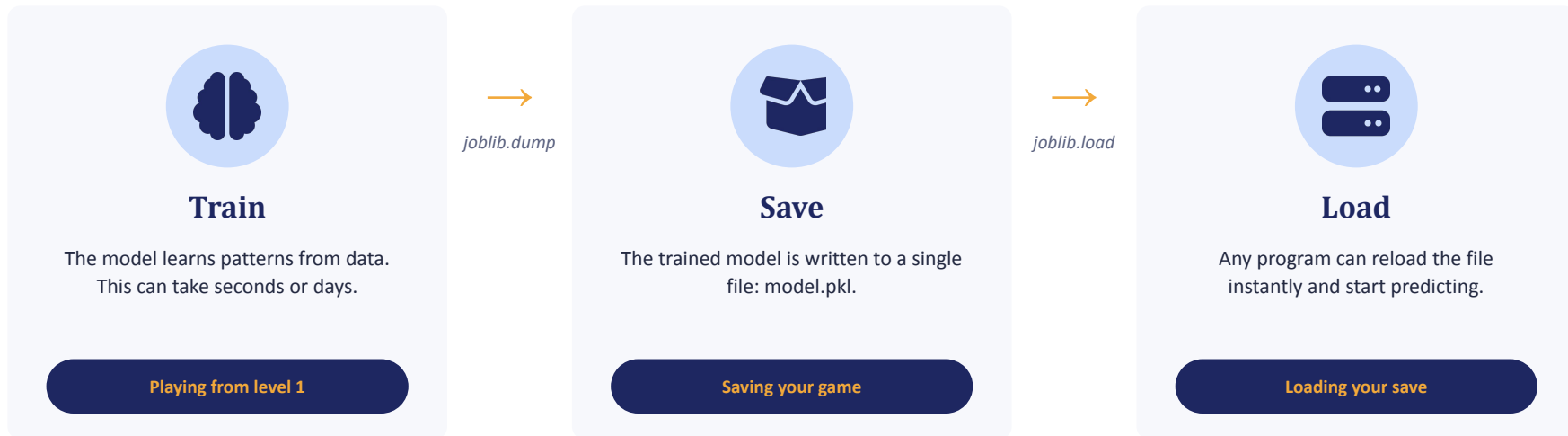
In production

- Runs on a server, around the clock
- Anyone with the URL can use it
- Any app can call it: web, mobile, other services
- Input is a request sent over the internet

Demo cue: predict in the notebook first, then send the same request to a live URL from your phone. That live URL is exactly what you will build tomorrow.

A Trained Model Is Just Data

Training is slow. Saving the result means you only ever do it once.



Why this matters: The API you build next does one thing at startup: it loads `model.pkl`. No retraining, ever, when a request comes in.

Export Your Model

Train once in the notebook, then save the result where the API can find it.

1**Open the notebook**

In Jupyter, open day1 → notebooks → 01_package_model.ipynb.

```
01_package_model.ipynb
```

2**Run the cells top to bottom**

Load data, train, and evaluate. You will see an accuracy score printed.

```
Shift + Enter    (runs one cell)
```

3**Export the model**

The last code cell saves your trained model to a file.

```
joblib.dump(model, "../api/model.pkl")
```

4**Check the file exists**

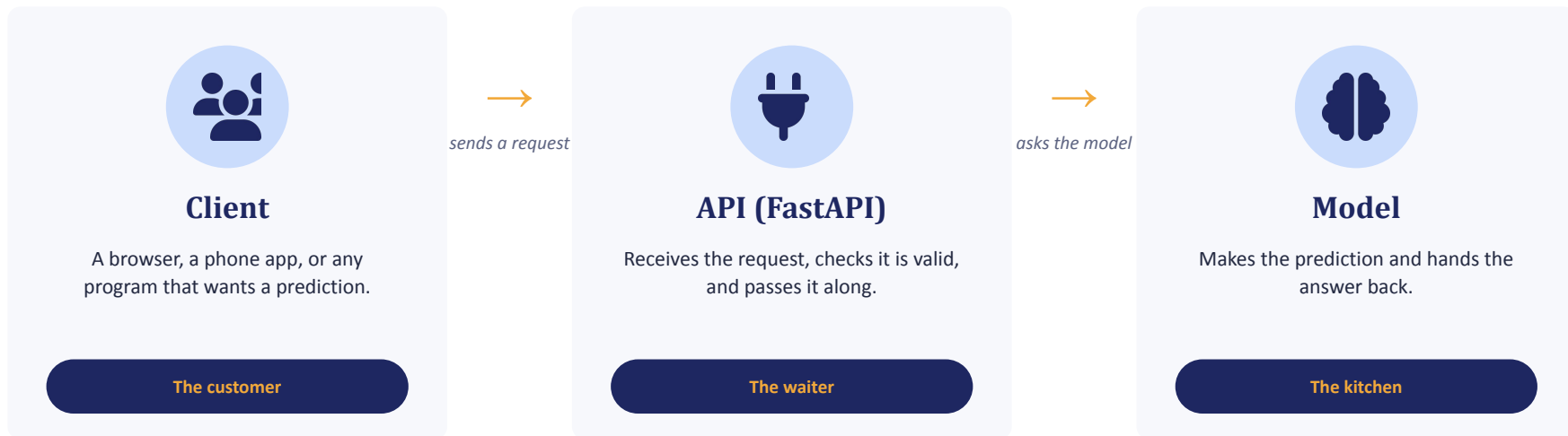
In Jupyter's file browser, open day1 → api.

```
Look for: model.pkl
```

Tip: No model.pkl in day1/api? Re-run the export cell. The API in the next session cannot start without it.

What Is an API? Think Restaurant

An API is how one program asks another program to do something.



Why bother? Anyone, on any device, in any language, can use your model without installing Python or seeing your code.

HTTP in 60 Seconds

Three ideas cover almost everything you need today.



GET: ask for something

You send nothing but the address. The server sends something back, just like opening a web page.

EXAMPLE

```
GET /health
```



POST: send data, get an answer

You include data in the request. The server uses it, here to make a prediction, and replies.

EXAMPLE

```
POST /predict
```



JSON and status codes

Data travels as JSON: plain text with names and values. Every reply has a code: 200 means OK, 422 means invalid input, 503 means not ready.

EXAMPLE

```
{"features": [5.1, 3.5, 1.4, 0.2]}
```

Two Small Wins

Build the API in two steps. Each one ends with something visibly working.

WIN 1

GET /health

Your API is alive and the model loaded

YOU SHOULD SEE

```
{"status": "ok", "model_loaded": true}
```

WIN 2

POST /predict

It makes real predictions

YOU SEND

```
{"features": [5.1, 3.5, 1.4, 0.2]}
```

YOU GET BACK

```
{"prediction": 0, "class_name": "setosa"}
```

How to test: click "Try it out" on the /docs page. No extra tools needed.

Run and Test Your API

Start the server, open the docs page, and try both endpoints.

1**Go to the API folder**

In a terminal, from the repo's main folder.

```
cd day1/api
```

2**Start the server**

You should see "Uvicorn running on http://127.0.0.1:8000".

```
uvicorn main:app --reload
```

3**Open the interactive docs**

Use "Try it out" on each endpoint. Win 1, then Win 2.

```
http://127.0.0.1:8000/docs
```

4**Break it on purpose**

Send only two numbers to /predict. What status code comes back?

```
{"features": [5.1, 3.5]}
```

Tip: Leave this code as it is. Tomorrow, this exact API is what goes live on the internet — no changes needed.

If Something Breaks

The four errors you are most likely to meet today. Flip back here anytime.

WHAT YOU SEE	WHAT IT MEANS	HOW TO FIX IT
<code>ModuleNotFoundError: No module named 'fastapi'</code>	Python cannot find a library	Run <code>pip install -r requirements.txt</code> , and make sure your virtual environment is activated
<code>"model_loaded": false</code> or <code>503 Model not loaded</code>	<code>model.pkl</code> is not in <code>day1/api</code>	Re-run the export cell in the notebook, then restart the server
<code>port is already allocated</code> / <code>address already in use</code>	Another program is using port 8000	Stop the other server (Ctrl + C), or run with <code>--port 8001</code> and browse to that port
<code>'uvicorn' is not recognized</code> / <code>command not found: uvicorn</code>	Libraries were not installed, or the wrong terminal is active	Re-run <code>pip install -r requirements.txt</code> in this terminal



END OF DAY 1 CHECKLIST

Before you leave today, you should have...



A trained model saved as `day1/api/model.pkl`



A FastAPI app where `/health` and `/predict` both work at `/docs`



Comfortable reading a request and a response written in JSON



Stuck on any of these? Flag it now — we'll fix it before Day 2 builds on top of it.

04

Day 2 — Ship It, and the Bigger Picture

Deploying yesterday's API live on Render, then zooming out to how the industry goes further

Day 2 Roadmap

3 hours · 4 blocks · 10-min break after deployment · ~55 min built-in buffer

40 min



Deploying to Render

What actually happens when you deploy. Hands-on: push to GitHub, connect Render, get a live URL.

20 min



Monitoring & Logging

Watch real requests hit your live API in Render's log viewer. Mostly a demo, light hands-on.

25 min



Wrap-Up Project

Make the API your own: tweak it, redeploy, and confirm the change went live.

30 min

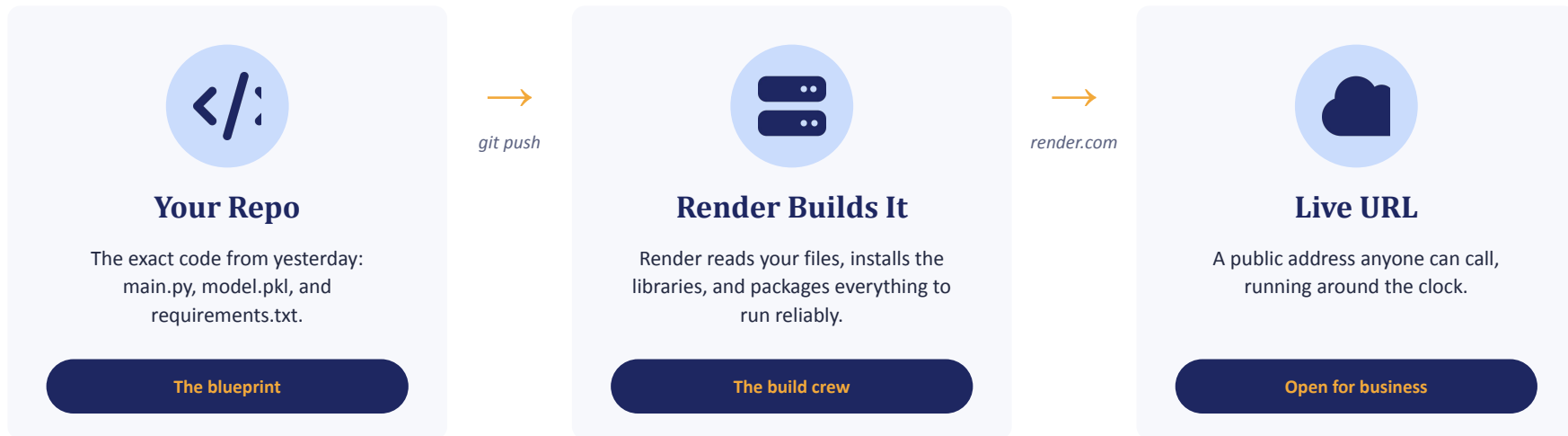


The Bigger Picture

Everything the industry does beyond today, named and explained. Nothing here to build.

What Happens When You Deploy?

You are not sending Render a running program. You are sending it instructions to build one.



One term worth knowing: Behind the scenes, Render often packages your app into something called a container — a way of bundling code so it runs identically everywhere. You will not build one by hand today; Render does it for you. More on this in The Bigger Picture.

Deploy Your API

Push your code, connect it to Render, and get a public URL.

1**Push your repo to GitHub**

If you have not already, commit and push everything from Day 1.

```
git push origin main
```

2**Create a new Web Service on Render**

Go to render.com, sign in, and click New + Web Service.

```
render.com/dashboard
```

3**Connect your repository**

Pick your GitHub repo. Render auto-detects it is a Python app.

```
Root directory: day1/api
```

4**Deploy and wait**

The first build takes a few minutes. Watch the build log scroll by.

```
Status: Building -> Live
```

5**Test your live URL**

Open /docs on your new URL, exactly like yesterday, just public now.

```
https://your-app.onrender.com/docs
```

Tip: Free-tier apps sleep after 15 minutes of no traffic. The first request after that takes about a minute to wake up. That is normal, not broken.

If Deployment Breaks

The most common Render hiccups, and what they actually mean.

WHAT YOU SEE	WHAT IT MEANS	HOW TO FIX IT
<code>Build failed / ModuleNotFoundError during build</code>	A library is missing from requirements.txt	Check requirements.txt lists every import your code uses, then redeploy
<code>Deploy succeeded, but the URL shows "Not Found"</code>	Render is looking in the wrong folder	Confirm the root directory is set to day1/api in the service settings
<code>The first request after a while is very slow</code>	The free tier spun the app down from inactivity	This is expected — wait about a minute, it wakes up on its own
<code>Everything is green, but /predict returns 503</code>	model.pkl was not committed to the repo	Check .gitignore is not excluding it, then git add, commit, and push again

Watch Your API Think

A few lines of logging turn a silent API into one you can actually observe.

```
logging.basicConfig(level=logging.INFO)
```

→ Turn logging on, and set how much detail to show

```
@app.middleware("http")
```

→ Run this code around every single request, automatically

```
start = time.perf_counter()
```

→ Note the time right before the request is handled

```
response = await call_next(request)
```

→ Let the request actually happen

```
latency_ms = (time.perf_counter() - start) * 1000
```

→ Work out how long that took, in milliseconds

```
logger.info(json.dumps({...}))
```

→ Write it all down: path, status code, and latency

See It Live

Send a few requests, then watch them appear in real time.



On your Render dashboard, open the Logs tab for your service.



Go back to your `/docs` page and send two or three `/predict` requests.



Watch each request appear in the log within a second or two — path, status, latency.



Now send an invalid request on purpose. Find the 422 in the log.

Make It Yours

One small change, redeployed, so you feel the full loop end to end.

1**Pick one small change**

Add a new field to the response, or change a message string in main.py.

```
# e.g. add "model_version": "v1"
```

2**Test it locally first**

Same as Day 1 — run uvicorn and check /docs before touching Render.

```
uvicorn main:app --reload
```

3**Push the change**

Render redeploys automatically whenever you push to GitHub.

```
git add . && git commit -m "update" && git push
```

4**Confirm it went live**

Refresh your live /docs page and see your change on the internet.

```
https://your-app.onrender.com/docs
```

Tip: This loop — change, test locally, push, confirm live — is the actual day-to-day rhythm of working on a production system.



END OF DAY 2 CHECKLIST

By now, you have...



A model deployed and reachable at a public URL



Watched real requests appear in a live log, including a failed one



Pushed a change and confirmed it went live



The ability to explain: package → API → deploy → monitor, start to finish



05

The Bigger Picture

Everything else the industry does with ML in production — named and explained, nothing here to build today

What We Used vs. What Else Exists

The middle column is what you touched. The right column is what you will hear at work.

STAGE	WHAT WE USED	WHAT YOU'LL SEE IN THE REAL WORLD
 Model Training	scikit-learn	TensorFlow, PyTorch, XGBoost
 Packaging	joblib / pickle	ONNX, MLflow Model Registry
 Serving / API	FastAPI	Flask, TorchServe, TF Serving
 Containerizing	(Render did this automatically)	Docker, Podman, Buildpacks — usually done by hand
 Deployment	Render	AWS (ECS/SageMaker), Azure ML, GCP Vertex AI
 Monitoring	Python logging	Datadog, Prometheus + Grafana, CloudWatch

The Full MLOps Maturity Map

You built the foundation. Real production teams layer these practices on top of it over time — not in a weekend.



YOU ARE HERE

Foundation — What You Built

Packaging a model, wrapping it in an API, deploying it live, and watching basic logs.



NEXT

Automation

Pipelines that automatically retest, retrain, and redeploy when code or data changes (CI/CD/CT), often using containers like Docker.



Data & Model Governance

Tracking exactly which data and model version is live in production (feature stores, model registry).



Safe Rollout & Advanced Monitoring

Testing a new model against the current one before fully switching, and catching drift automatically (Champion/Challenger).



Scale & Security

Making sure the system holds up under real traffic and stays secure (load testing, autoscaling, access control).

Where to Go From Here

The concepts transfer directly — only the tool names change.



Major Clouds

AWS SageMaker / ECS, Azure ML, GCP Vertex AI — same steps, managed infrastructure



Orchestration

Kubernetes for running many containers reliably at scale



MLOps Tooling

MLflow or Weights & Biases for experiment + model tracking



Observability

Prometheus + Grafana, Datadog, or CloudWatch for production-grade monitoring



You Shipped It.

From notebook to a live, monitored API — in two days. Thank you for building with us.

Questions? Keep them coming — the best learning happens after the workshop ends.

Jimmi Kr. Bharti · [linkedin.com/in/jimmi-kr](https://www.linkedin.com/in/jimmi-kr)